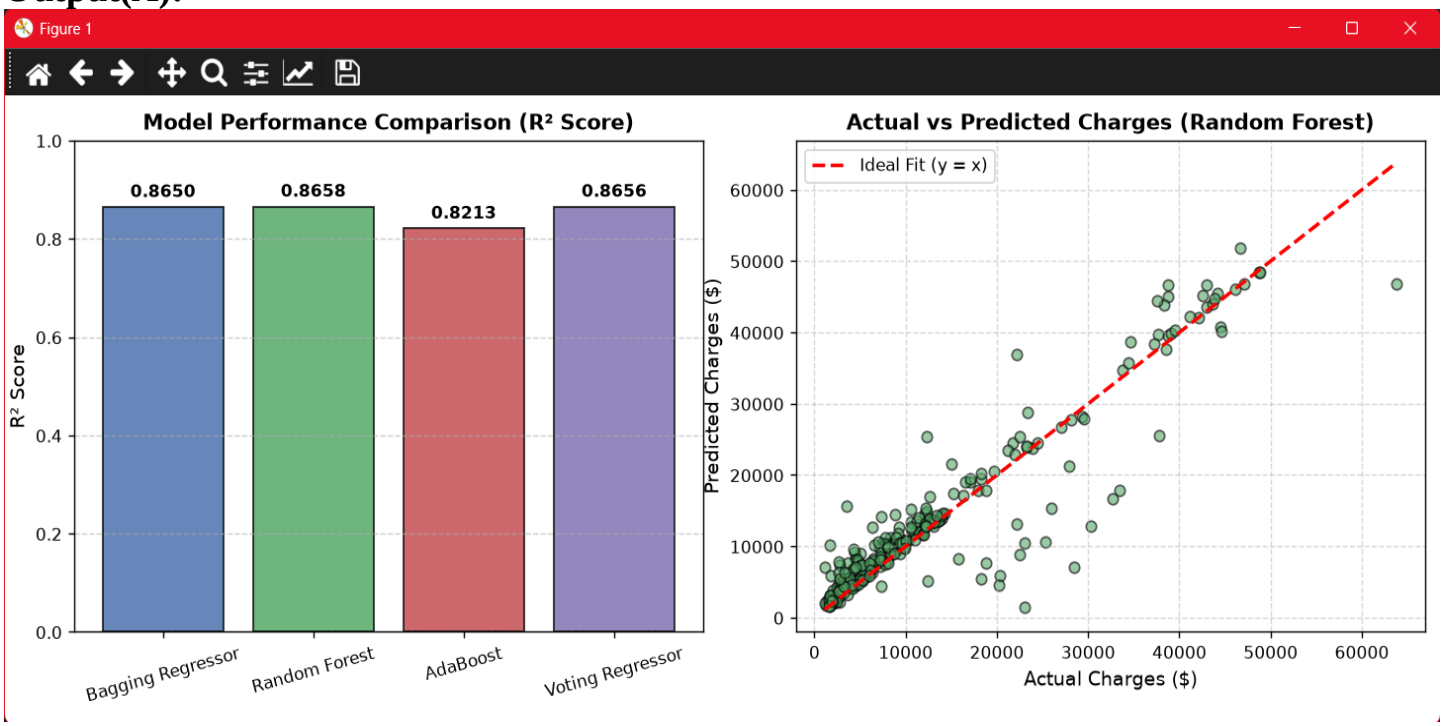
Worksheet 5

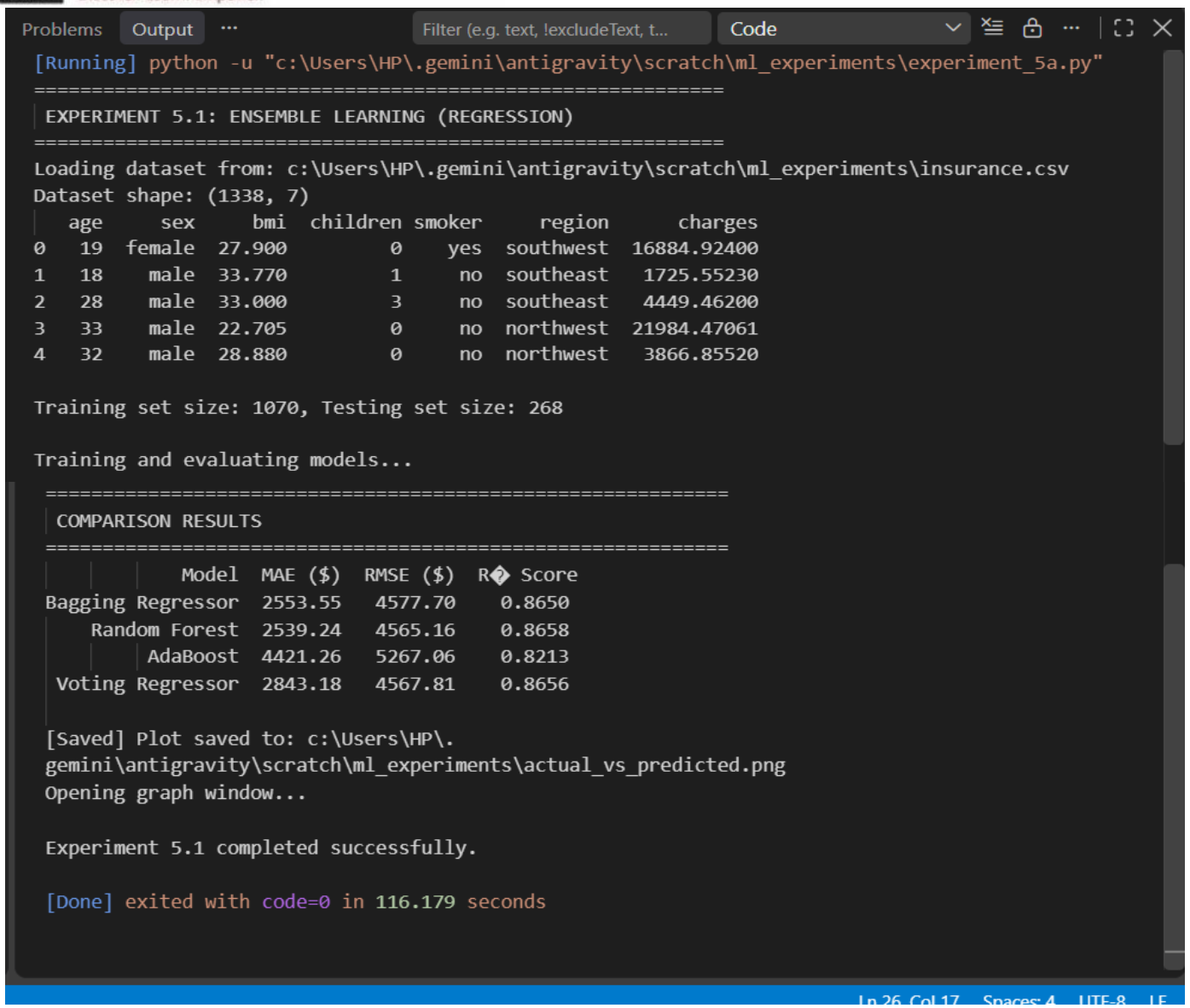
- **Aim(A):** Implement any one ensemble learning techniques such as Bagging, Random Forest, AdaBoost, or Voting Classifier using a suitable dataset. Compare their performance using appropriate evaluation metrics and discuss the results.
- **Objective(A):** To implement an ensemble learning technique such as Bagging, Random Forest, AdaBoost, or Voting Classifier on a suitable dataset, evaluate its performance using metrics such as accuracy, precision, recall, and F1-score, and compare the results to understand the effectiveness of ensemble learning.
- **Code(A):**

```
import os
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder, StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import BaggingRegressor, RandomForestRegressor, AdaBoostRegressor,
VotingRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
# Ensure file path is resolved relative to the script location
script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, 'insurance.csv')
print("=" * 60)
print(" EXPERIMENT 5.1: ENSEMBLE LEARNING (REGRESSION)")
print("=" * 60)
numerical_cols = ['age', 'bmi', 'children']
# Preprocessing pipeline
preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numerical_cols),
        ('cat', OneHotEncoder(drop='first'), categorical_cols)
    ]
)
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
print(f"\nTraining set size: {X_train.shape[0]}, Testing set size: {X_test.shape[0]}")
 (results)
print("\n" + "=" * 60)
print(" COMPARISON RESULTS ")
print("=" * 60)
```

```
print(results_df.to_string(index=False))
# Subplot 1: R² Score Comparison
bars = ax1.bar(results_df["Model"], results_df["R² Score"], color=['#4C72B0', '#55A868', '#C44E52',
'#8172B2'], edgecolor='k', alpha=0.85)
ax1.set_ylabel('R² Score', fontsize=11)
ax1.set_title('Model Performance Comparison (R² Score)', fontsize=12, fontweight='bold')
ax1.set_ylim(0, 1.0)
ax1.grid(axis='y', linestyle='--', alpha=0.6)
for bar in bars:
    height = bar.get_height()
    ax1.annotate(f'{height:.4f}',
                 xy=(bar.get_x() + bar.get_width() / 2, height),
                 xytext=(0, 3),
                 textcoords="offset points",
                 ha='center', va='bottom', fontweight='bold', fontsize=10)
ax2.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--', lw=2, label='Ideal Fit (y = x)')
ax2.set_xlabel('Actual Charges ($)', fontsize=11)
ax2.set_ylabel('Predicted Charges ($)', fontsize=11)
ax2.set_title('Actual vs Predicted Charges (Random Forest)', fontsize=12, fontweight='bold')
ax2.grid(True, linestyle='--', alpha=0.5)
ax2.legend()
plt.tight_layout()
plt.savefig(plot_path, dpi=300)
print(f"\n[Saved] Plot saved to: {plot_path}")
print("Opening graph window...")
plt.show()
print("\nExperiment 5.1 completed successfully.")
```

- **Output(A):**





- **Learning Outcome(A)**:
  1. Understand the concept of ensemble learning.
  2. Implement techniques such as Bagging, Random Forest, AdaBoost, or Voting Classifier.
  3. Apply ensemble models to a real-world dataset.
  4. Evaluate model performance using accuracy, precision, recall, and F1-score.

- **Aim (B):** Write a Python program to solve a suitable real-world problem using the K-Means clustering algorithm. Select an appropriate dataset, determine the optimal number of clusters, visualize the clusters, and discuss the outcomes.

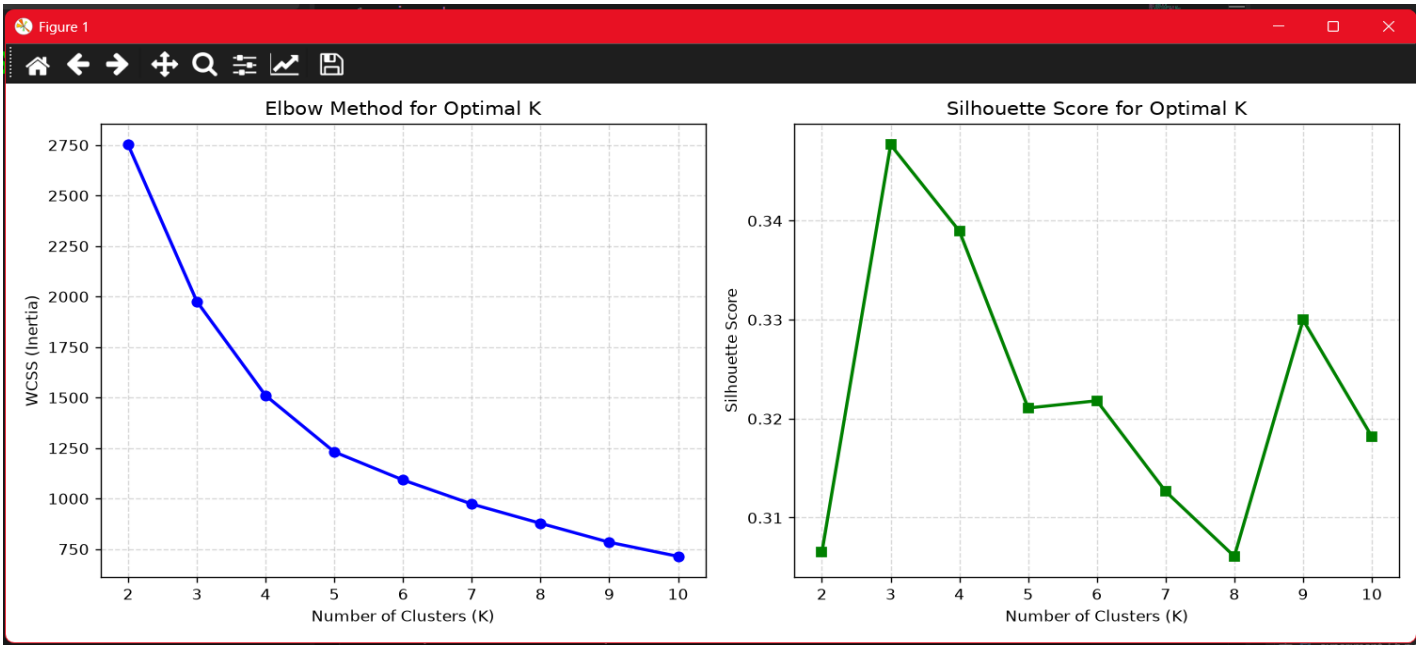
- **Objective(B):** To develop a Python program using the K-Means clustering algorithm to solve a suitable realworld problem, determine the optimal number of clusters, visualize the resulting clusters, and analyze the clustering outcomes.

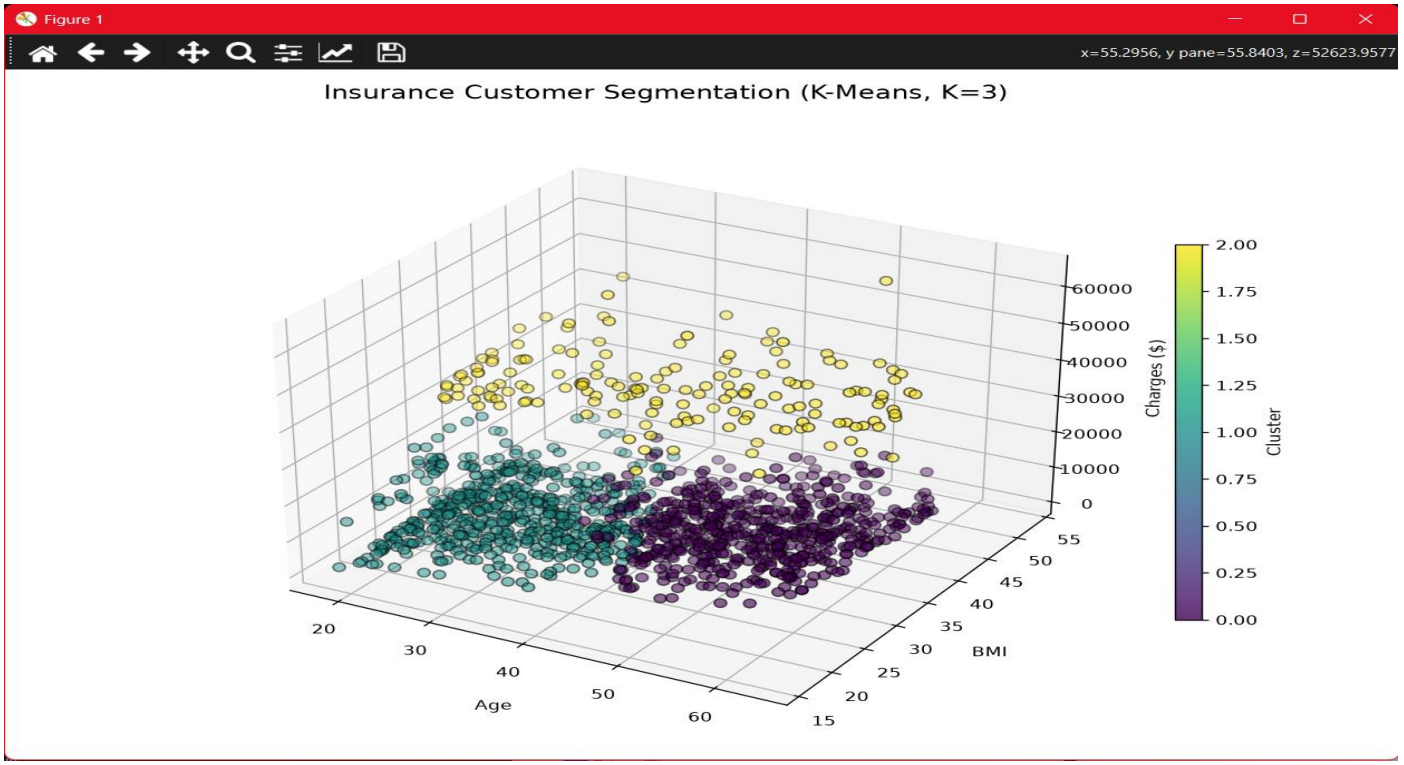
- **Code(B):**

```
import os
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score
# Ensure file path is resolved relative to script location
script_dir = os.path.dirname(os.path.abspath(__file__))
csv_path = os.path.join(script_dir, "insurance.csv")
print("=" * 60)
print(" EXPERIMENT 5.2: K-MEANS CLUSTERING (CUSTOMER SEGMENTATION)")
print("=" * 60)
# Load dataset
print(f"Loading dataset from: {csv_path}")
df = pd.read_csv(csv_path)
print("Dataset loaded successfully!")
print(f"Shape: {df.shape}")
print(df.head())
# Select features
X = df[["age", "bmi", "charges"]]
# Scale data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# Find best K
plt.grid(True, linestyle="--", alpha=0.5)
ax.set_ylabel("BMI", labelpad=10)
ax.set_zlabel("Charges ($)", labelpad=10)
ax.set_title("Insurance Customer Segmentation (K-Means, K=3)", fontsize=14, pad=15)
fig.colorbar(scatter, ax=ax, label="Cluster", shrink=0.6)
plt.tight_layout()
cluster_3d_path = os.path.join(script_dir, "kmeans_3d_clusters.png")
plt.savefig(cluster_3d_path, dpi=300)
print(f"[Saved] 3D Cluster visualization saved to: {cluster_3d_path}")
plt.show()
# Display cluster profiles and interpretations
result = df.groupby("Cluster")[["age", "bmi", "charges"]].mean().round(2)
print("\n" + "=" * 60)
print(" CLUSTER PROFILES (MEAN VALUES) ")
print("=" * 60)
print(result)
print("\n" + "=" * 60)
print(" CUSTOMER COUNT PER CLUSTER ")
print("=" * 60)
print(df["Cluster"].value_counts().sort_index())
print("\n" + "=" * 60)
print(" CLUSTER INTERPRETATION & DISCUSSION ")
print("=" * 60)
for cluster_id in sorted(df["Cluster"].unique()):
```

```
    print(f"  - Avg Age: {avg_age:.1f} yrs | Avg BMI: {avg_bmi:.1f} | Avg Charges: ${avg_charges:,.2f}")
    if avg_charges > 25000:
        print("  - Profile: High-Risk / High-Cost segment (typically smokers and high medical costs)")
    elif avg_bmi > 30 and avg_age > 40:
        print("  - Profile: Moderate-to-High Risk older adults with elevated BMI")
    else:
        print("  - Profile: Low-Risk / Younger / Low-Cost demographic with baseline health costs")
print("=" * 60)
```

- **Output(B):**





```
Problems  Output  ...   Filter (e.g. text, !excludeText, t...   Code

[Running] python -u "c:\Users\HP\.gemini\antigravity\scratch\ml_experiments\experiment_5b.py"
============================================================
 EXPERIMENT 5.2: K-MEANS CLUSTERING (CUSTOMER SEGMENTATION)
============================================================
Loading dataset from: c:\Users\HP\.gemini\antigravity\scratch\ml_experiments\insurance.csv
Dataset loaded successfully!
Shape: (1338, 7)
   age     sex     bmi  children smoker     region      charges
0   19  female  27.900         0    yes  southwest  16884.92400
1   18    male  33.770         1     no  southeast   1725.55230
2   28    male  33.000         3     no  southeast   4449.46200
3   33    male  22.705         0     no  northwest  21984.47061
4   32    male  28.880         0     no  northwest   3866.85520

Calculating WCSS and Silhouette Scores for K = 2 to 10...
  K =  2 | WCSS =    2750.61 | Silhouette Score = 0.3065
  K =  3 | WCSS =    1974.95 | Silhouette Score = 0.3477
  K =  4 | WCSS =    1509.23 | Silhouette Score = 0.3389
  K =  5 | WCSS =    1230.19 | Silhouette Score = 0.3211
  K =  6 | WCSS =    1091.13 | Silhouette Score = 0.3218
  K =  7 | WCSS =     972.07 | Silhouette Score = 0.3126
  K =  8 | WCSS =     876.55 | Silhouette Score = 0.3061
  K =  9 | WCSS =     782.84 | Silhouette Score = 0.3300
  K = 10 | WCSS =     713.06 | Silhouette Score = 0.3182

[Saved] Elbow & Silhouette plot saved to: c:\Users\HP\.
gemini\antigravity\scratch\ml_experiments\kmeans_elbow_silhouette.png

Fitting K-Means with K = 3...
[Saved] 3D Cluster visualization saved to: c:\Users\HP\.
gemini\antigravity\scratch\ml_experiments\kmeans_3d_clusters.png

============================================================
 CLUSTER PROFILES (MEAN VALUES)
============================================================
              age    bmi   charges
Cluster
0         51.60  31.11  12786.56
1         27.33  29.02   6425.08
2         39.79  35.26  40712.65

============================================================
 CUSTOMER COUNT PER CLUSTER
Cluster
0    572
1    605
2    161
Name: count, dtype: int64

============================================================
 CLUSTER INTERPRETATION & DISCUSSION
============================================================
Cluster 0 (N=572):
  - Avg Age: 51.6 yrs | Avg BMI: 31.1 | Avg Charges: $12,786.56
  - Profile: Moderate-to-High Risk older adults with elevated BMI
Cluster 1 (N=605):
  - Avg Age: 27.3 yrs | Avg BMI: 29.0 | Avg Charges: $6,425.08
  - Profile: Low-Risk / Younger / Low-Cost demographic with baseline health costs
Cluster 2 (N=161):
  - Avg Age: 39.8 yrs | Avg BMI: 35.3 | Avg Charges: $40,712.65
  - Profile: High-Risk / High-Cost segment (typically smokers and high medical costs)
============================================================

[Done] exited with code=0 in 237.676 seconds
```

Learning Outcome(B):

1. Understand the concept of K-Means clustering.
2. Apply K-Means to a real-world dataset.
3. Determine the optimal number of clusters using suitable methods such as the Elbow Method.
4. Visualize and interpret the formed clusters.